

Requisitos de software: documentación, validación y versionamiento

Breve descripción:

Este componente formativo fortalece las capacidades para analizar y levantar requisitos de software mediante métodos y estándares técnicos reconocidos. Se tendrá con él las bases para recolectar información de los stakeholders, documentar y validar requisitos de forma clara y estructurada, asegurando que respondan a las necesidades del cliente y sirvan como base para soluciones de software exitosas.

Abril 2026

Tabla de contenido

Introducción	3
1. Conceptos fundamentales de la ingeniería de requisitos	6
2. Tipos de documentación de requisitos.....	9
2.1. Lenguaje natural	9
2.2. Modelos conceptuales.....	21
2.3. Enfoque híbrido	35
3. Buenas prácticas de documentación.....	39
4. Informe de requisitos.....	45
5. Historias de usuario	50
6. Listas de chequeo para validación de información	55
7. Técnicas para validar requisitos.....	59
8. Versionamiento de requisitos: gestión de cambios	63
9. Herramientas y tecnologías para la gestión de requisitos	65
Síntesis	68
Glosario	70
Referencias bibliográficas	72
Créditos	73

Introducción

Imagine que a un profesional se le encarga dirigir un proyecto para desarrollar un sistema de información para una empresa de distribución de alimentos. El gerente expresa una visión general: “mejorar los procesos y aumentar la competitividad”. Sin embargo, durante las reuniones iniciales surgen múltiples expectativas: automatización de inventarios, fortalecimiento de la relación con los clientes, localización de productos en tiempo real y generación de reportes precisos para auditoría. Ante este panorama, se hace evidente la necesidad de identificar con claridad los verdaderos requisitos del sistema, su propósito fundamental y la prioridad de sus funcionalidades.

Este escenario es frecuente en el desarrollo de software. Diversos estudios industriales indican que entre el 70 % y el 80 % de los defectos en los sistemas se originan en errores o deficiencias durante la captura y documentación de requisitos. El problema no radica en la falta de habilidades de programación, sino en la construcción de soluciones basadas en requisitos mal entendidos o incompletos.

La ingeniería de requisitos aborda precisamente esta problemática. Se trata de un proceso sistemático y disciplinado para desarrollar, analizar, comunicar y mantener los requisitos a lo largo del ciclo de vida del proyecto. Esta disciplina integra técnicas de análisis, comunicación efectiva, conocimiento técnico y pensamiento crítico, y va más allá de la simple redacción de especificaciones.

En el contexto del desarrollo de software, un requisito es una declaración clara de lo que el sistema debe hacer. Para que sea correcto, debe ser específico, verificable, realizable y trazable. Por ejemplo, afirmar que un sistema debe ser “rápido” no constituye un requisito verificable; en cambio, establecer que debe responder a

consultas de inventario en menos de dos segundos en el 95 % de los casos permite su medición, prueba y validación.

Partiendo de lo anterior, se invita a acceder al siguiente video, el cual contextualiza y articula la temática que se abordará durante este componente formativo:

Video 1. Requisitos de software: documentación, validación y versionamiento



[Enlace de reproducción del video](#)

Transcripción del video: Requisitos de software: documentación, validación y versionamiento

En el desarrollo de software actual, comprender qué se debe construir y por qué es tan importante como conocer cómo hacerlo. Por ello, este componente formativo introduce al aprendiz en el campo estratégico de la ingeniería de requisitos, una disciplina fundamental para el éxito de los proyectos de software.

A lo largo del contenido, se evidencia que documentar requisitos correctamente no es una tarea administrativa, sino una actividad crítica que influye directamente en la calidad, el costo y el cumplimiento de los objetivos del proyecto. Requisitos mal definidos suelen derivar en retrabajos, sobre costos y soluciones que no satisfacen al cliente.

El componente presenta diversas formas de documentar requisitos, desde el lenguaje natural hasta el uso de modelos y diagramas estructurados, así como estándares reconocidos internacionalmente que orientan buenas prácticas de documentación. También se analizan las historias de usuario, destacando su valor para expresar necesidades desde la perspectiva del usuario y facilitar la comunicación en entornos ágiles.

Asimismo, se abordan técnicas de validación que permiten confirmar que los requisitos reflejan las necesidades reales del negocio y se explica cómo gestionar la evolución de los requisitos a lo largo del ciclo de vida del software mediante procesos controlados de cambio.

Más allá de métodos y formatos, el componente promueve una mentalidad propia del ingeniero de requisitos: hacer las preguntas correctas, escuchar a los stakeholders, identificar ambigüedades y comunicar de forma clara entre actores técnicos y no técnicos, contribuyendo así a soluciones de software pertinentes y de calidad.

1. Conceptos fundamentales de la ingeniería de requisitos

Para entender adecuadamente la ingeniería de requisitos, se necesita empezar por los conceptos más fundamentales. La pregunta más básica es:

¿Qué es un requisito? Un requisito es una condición o capacidad que un usuario necesita para resolver un problema o alcanzar un objetivo.

En el contexto del desarrollo de software, es una declaración de lo que el sistema de software debe hacer o una restricción sobre cómo debe hacerlo.

Es importante distinguir entre necesidades, requisitos y características:

- **Necesidad**

Es un problema o una carencia que alguien experimenta. Por ejemplo, un banco tiene la necesidad de procesar transacciones más rápidamente para mejorar la satisfacción del cliente.

- **Requisito**

Es la traducción de esa necesidad en términos que pueden ser implementados. Por ejemplo, el sistema debe procesar transacciones bancarias estándar en menos de 3 segundos.

- **Característica**

Es la implementación específica de ese requisito en el software. Por ejemplo, la arquitectura del sistema utiliza bases de datos distribuidas y mecanismos de caché para lograr esa velocidad.

Ahora bien, ¿por qué es tan crítica la ingeniería de requisitos? Considere el costo de corregir un defecto en diferentes fases del proyecto:

- **Captura de requisitos:** un defecto identificado durante esta fase puede ser corregido simplemente modificando un documento, quizás con un costo de unos cientos de pesos.
- **Fase de diseño:** el mismo defecto, si no es identificado hasta esta fase, puede costar miles de pesos en rediseño.
- **Fase de implementación:** si persiste hasta la implementación, el costo es aún mayor.
- **Fase de pruebas:** y si permanece hasta esta fase o peor aún, hasta que el sistema está en producción, el costo puede ser de millones de pesos, más los costos de reputación y satisfacción del cliente.

Estudios de la Universidad de Michigan han mostrado que corregir un error en requisitos durante la fase de desarrollo cuesta en promedio 10 veces más que corregirlo durante la fase de especificación. Y si el error se descubre después de que el software está en producción, puede costar 100 veces más. Esta es la razón por la cual invertir tiempo y cuidado en la ingeniería de requisitos no es un lujo, es una necesidad económica.

La ingeniería de requisitos no es simplemente "hablar con el cliente"; es un proceso riguroso que involucra múltiples actividades: elicitación (descubrimiento de requisitos), análisis (comprensión profunda), especificación (documentación formal), validación (verificación de que los requisitos son correctos) y gestión (manejo de cambios). Cada una de estas actividades requiere habilidades específicas y herramientas apropiadas.

Un aspecto importante a entender es que hay diferentes tipos de requisitos:

a) Requisitos funcionales

Describen qué debe hacer el sistema. Por ejemplo: "el sistema debe permitir que los usuarios registren nuevas órdenes de compra".

b) Requisitos no funcionales

Describen cómo debe funcionar el sistema o qué limitaciones debe respetar. Por ejemplo: "el sistema debe estar disponible 99.9 % del tiempo" o "el sistema debe procesar hasta 10.000 transacciones por hora".

c) Requisitos de restricción

Describen límites físicos o normativas. Por ejemplo: "el sistema debe cumplir con la ley de protección de datos GDPR" o "el sistema debe funcionar en dispositivos móviles con conectividad 4G lenta".

Finalmente, es crucial entender que los requisitos no son descubiertos, son elicitados. El cliente frecuentemente no sabe exactamente qué necesita. Tiene una intuición, una sensación de que algo debe mejorar, pero no tiene una especificación clara. Es el trabajo del ingeniero de requisitos ayudar al cliente a articular esa intuición en requisitos concretos. Esto requiere escucha activa, preguntas profundas, así como la capacidad de ver patrones y conexiones que el cliente mismo no ve. En esencia, el ingeniero de requisitos es un traductor cultural que convierte las visiones y necesidades del negocio en especificaciones técnicas que los desarrolladores pueden implementar.

2. Tipos de documentación de requisitos

Una de las primeras decisiones que debe tomar un equipo de ingeniería de requisitos es cómo documentarlos. Esta es una decisión fundamental porque la forma en que se documentan los requisitos afecta profundamente cómo se comunican, se entienden y se implementan. Existen múltiples opciones, cada una con sus ventajas y limitaciones. La clave es entender el contexto y elegir sabiamente.

2.1. Lenguaje natural

Es la forma de comunicación humana cotidiana: español, inglés, francés, entre otros idiomas. En contraste con los lenguajes formales o de programación, el lenguaje natural no está regido por una gramática completamente rígida ni por símbolos matemáticos estrictos.

A continuación, se presenta un pódcast que explora el uso del lenguaje natural en la documentación de requisitos, explicando cómo esta forma de comunicación conecta a usuarios y desarrolladores, y los desafíos que plantea para asegurar claridad y precisión en los proyectos de software:

Transcripción del pódcast. Lenguaje natural
Hombre: todo proyecto de software empieza mucho antes de la primera línea de código. Es una conversación donde alguien describe una necesidad y otra persona intenta entenderla para transformarla en algo que

Transcripción del podcast. Lenguaje natural

pueda construirse. Y en esa conversación, la herramienta principal es el lenguaje.

Mujer: y ese lenguaje, que en la vida cotidiana funciona sin problema, adquiere un valor distinto cuando se convierte en la base de algo que después debe ejecutarse con precisión.

Hombre: ese es el valor del lenguaje natural como herramienta principal para capturar requisitos. Es la forma más común de documentar lo que un sistema debe hacer, porque no requiere conocimientos técnicos y permite que cualquier persona involucrada pueda leerlo y entenderlo de manera general.

Mujer: y ahí está el desafío. El lenguaje natural funciona bien para comunicar ideas generales, pero cuando se convierte en la base de decisiones técnicas, su ambigüedad empieza a pesar.

Hombre: por eso aparecen frases que se repiten constantemente en los requisitos de software. El sistema debe ser fácil de usar. Debe responder rápidamente. Debe ser seguro. Son expresiones que en principio parecen suficientes, porque todos creen entenderlas.

Mujer: sin embargo, cuando se observan con más detalle, cada una de esas palabras abre un rango de interpretaciones bastante amplio. Fácil de usar no es una sola definición. Puede significar que cualquier persona

Transcripción del pódcast. Lenguaje natural

pueda aprender sin capacitación, o que la interfaz sea intuitiva, o que el sistema no genere confusión durante su uso.

Hombre: con la rapidez ocurre algo similar. Lo que para un desarrollador puede ser una respuesta aceptable dentro de ciertos parámetros técnicos, para un usuario puede ser insuficiente si no coincide con su expectativa de inmediatez. No hay un valor único mientras no se defina explícitamente.

Mujer: y la seguridad también entra en ese mismo terreno. Puede referirse a protección de datos, control de accesos, encriptación, respaldo de información o cumplimiento de estándares. Todo eso puede estar contenido en una sola palabra si no se detalla qué se espera exactamente.

Hombre: el punto crítico es que estas diferencias no generan problemas en el momento de la conversación. Mientras se discuten, todo parece avanzar con normalidad. El lenguaje permite continuar sin detener el proceso, y cada persona trabaja con la interpretación que considera correcta.

Mujer: pero cuando esas interpretaciones empiezan a convertirse en decisiones de diseño y desarrollo, el sistema comienza a tomar forma desde una base que no es completamente compartida. Y en ese momento, pequeñas diferencias iniciales empiezan a crecer sin que sea evidente de inmediato.

Transcripción del podcast. Lenguaje natural

Hombre: el desarrollo avanza, el sistema funciona y las pruebas técnicas pueden ser exitosas. Y aun así, cuando se compara el resultado con lo que alguien esperaba al inicio, puede aparecer una distancia entre lo construido y lo imaginado.

Mujer: lo importante ahí es entender que esa distancia no siempre viene de un error en la implementación. El sistema puede estar técnicamente bien construido. Lo que ocurrió es que lo que se entendió al inicio no era exactamente lo mismo para todos los involucrados.

Hombre: en el desarrollo de software, el lenguaje natural es la herramienta con la que todo empieza. Y precisamente por eso requiere cuidado. Su mayor virtud, que lo hace accesible para cualquier persona, puede generar interpretaciones distintas cuando no se gestiona con atención.

Mujer: porque la calidad de un sistema no depende únicamente de cómo se construye. Depende también de qué tan claramente se entendió lo que debía construirse desde el principio. (Pausa)

Gracias por su compañía. Nos escuchamos en el próximo programa.

El lenguaje natural, cuando se usa correctamente, puede ser poderoso. Por ejemplo, si especifica "el sistema debe procesar solicitudes de reembolso en menos de 30 segundos para el 95 % de los casos y en menos de 2 minutos para el 100 % de los

casos", eso es específico, verificable y orientado a resultados. El lenguaje natural es excelente para narrativa, contexto y para comunicar a personas sin formación técnica.

Por lo anterior, antes de crear diagramas UML, modelos conceptuales o cualquier artefacto técnico, la mayoría de los proyectos de software comienzan con conversaciones, entrevistas y documentos escritos en lenguaje natural. Esto ocurre porque:

- Los clientes y usuarios no conocen notaciones técnicas como UML o fórmulas matemáticas.
- El lenguaje natural es accesible para todos los involucrados en el proyecto (stakeholders).
- Permite capturar el contexto, las intenciones y las expectativas de forma rápida.
- Es flexible y puede describir situaciones complejas sin restricciones de formato.

Ejemplo práctico

Un cliente le dice al ingeniero de requisitos:

"Necesito que el sistema me permita registrar los pedidos de mis clientes, ver cuales están pendientes de entrega y enviarles un correo cuando el pedido haya sido despachado."

Esta frase en lenguaje natural ya contiene tres requisitos funcionales que el ingeniero debe capturar, analizar y documentar formalmente.

El lenguaje natural es una herramienta en la documentación de requisitos es el proceso de registrar formalmente lo que el sistema debe hacer (requisitos funcionales) y como debe hacerlo (requisitos no funcionales). El lenguaje natural es el medio más común para este proceso. La siguiente tabla explica un poco estos procesos:

Tabla 1. Tipo de requisitos que se documentan con lenguaje natural

Tipo de requisito	Descripción	Ejemplo en lenguaje natural
Funcional	Describe qué debe hacer el sistema: sus funciones y comportamientos.	El sistema debe permitir al usuario registrarse con correo y contraseña.
No funcional	Describe cómo debe comportarse el sistema: rendimiento, seguridad, usabilidad.	El sistema debe cargar la página principal en menos de 2 segundos.
De negocio	Define los objetivos y reglas del negocio que el sistema debe soportar.	Solo los usuarios mayores de 18 años pueden realizar compras en línea.
De restricción	Impone limitaciones técnicas, legales o de presupuesto.	El sistema debe ser desarrollado usando tecnologías de código abierto.
De usuario	Captura las necesidades desde la perspectiva del usuario final.	Como comprador, desea guardar los productos favoritos para revisarlos después.

Un buen requisito documentado en lenguaje natural debe seguir una estructura clara que evite ambigüedades. La fórmula más utilizada es:

Fórmula estándar para escribir un requisito

El sistema DEBE / DEBERÍA / PODRÍA + [acción] + [objeto/entidad] + [condición o restricción opcional]

Ejemplos:

- El sistema DEBE permitir al usuario iniciar sesión con usuario y contraseña.
- El sistema DEBERÍA enviar un correo de confirmación cuando se complete una compra.
- El sistema PODRÍA mostrar sugerencias de búsqueda mientras el usuario escribe.

La palabra clave (DEBE, DEBERÍA, PODRÍA) indica el nivel de obligatoriedad del requisito.

Las ventajas del lenguaje natural son las siguientes:

- **Accesibilidad:** cualquier persona puede leer y escribir requisitos sin formación técnica especial.
- **Expresividad:** permite capturar matices, contexto y casos excepcionales con facilidad.
- **Flexibilidad:** se adapta a cualquier dominio de negocio sin restricciones de sintaxis.
- **Comunicación directa:** facilita la comunicación entre clientes e ingenieros.
- **Velocidad:** permite documentar requisitos rápidamente en entrevistas o talleres.

También existen algunas limitaciones:

Tabla 2. Limitaciones del lenguaje natural

Limitación	Descripción del problema	Cómo superarla
Ambigüedad	Una misma frase puede interpretarse de distintas formas.	Usar términos precisos y definir un glosario del proyecto.
Inconsistencia	Dos requisitos pueden contradecirse entre sí.	Realizar revisiones cruzadas y usar matrices de trazabilidad.
Incompletitud	Se omiten casos bordes o escenarios excepcionales.	Hacer preguntas de validación y usar listas de verificación.
Verbosidad	Textos muy largos dificultan la lectura y el mantenimiento.	Usar plantillas y formatos estandarizados (IEEE 830).
Subjetividad	Términos como “rápido”, “fácil” o “seguro” son ambiguos.	Reemplazar con métricas concretas y criterios de aceptación.

Importante - El peligro de la ambigüedad

Ejemplo del requisito AMBIGUO:

"El sistema debe ser rápido".

Problemas:

¿Qué tan rápido? ¿Rápido para quién? ¿En qué condiciones?

Ejemplo de requisito CORRECTO (sin ambigüedad):

"El sistema debe procesar y mostrar los resultados de búsqueda en menos de 1 segundo para un mínimo de 500 usuarios concurrentes".

La versión correcta es medible, verificable y no da lugar a interpretaciones.

Existen dos grandes enfoques para usar el lenguaje natural en la documentación de requisitos:

a) Lenguaje natural no estructurado

Es el texto libre, sin un formato predefinido. Se usa en las primeras fases de captura de requisitos, como en entrevistas o actas de reunión. Su ventaja es la rapidez; su desventaja es que puede resultar difícil de analizar y verificar.

Ejemplo de lenguaje natural no estructurado

Extracto de acta de reunión con el cliente:

"El cliente mencionó que necesita que los vendedores puedan ver sus comisiones en tiempo real.

También comentó que le gustaría tener alertas cuando un cliente lleva más de 30 días sin comprar.

La gerencia quiere reportes mensuales que se puedan exportar a Excel".

Este tipo de texto necesita ser procesado y formalizado por el ingeniero de requisitos.

b) Lenguaje natural estructurado

Usa plantillas y formatos predefinidos para garantizar uniformidad, trazabilidad y facilidad de revisión. Es la forma recomendada para documentar requisitos formalmente.

Tabla 3. Plantilla típica de requisito estructurado

Campo	Descripción	Ejemplo
ID del Requisito.	Identificador único del requisito.	REQ-FUN-001.
Nombre.	Título corto y descriptivo.	Inicio de sesión de usuario.
Descripción.	Enunciado completo del requisito.	El sistema debe permitir al usuario autenticarse con correo electrónico y contraseña.
Prioridad.	Nivel de importancia del requisito.	Alta.
Criterio de aceptación.	Cómo se verifica que el requisito esta cumplido.	El usuario puede ingresar al sistema en menos de 3 intentos.
Stakeholder.	Quién solicitó o se ve afectado por el requisito.	Usuarios registrados, equipo de seguridad.
Fuente.	De dónde surgió el requisito.	Entrevista con el cliente - Acta 001.

El ingeniero de requisitos usa diversas técnicas para obtener información de los stakeholders y transformarla en requisitos bien documentados, entre las que se encuentran:

- **Entrevistas**

Consisten en conversaciones directas con usuarios, clientes y expertos del dominio. Son la técnica más utilizada y en ella, el ingeniero debe preparar preguntas abiertas y de cierre que guíen la conversación hacia los requisitos del sistema. Se encuentran:

- **Preguntas abiertas:** ¿Qué funciones debe tener el sistema? ¿Qué problemas resuelve?
- **Preguntas de profundización:** ¿Puede dar un ejemplo? ¿Qué pasa si...?
- **Preguntas de confirmación:** se entiende que el sistema debe... ¿Es correcto?

Talleres Joint Application Development (JAD)

Reúnen a todos los stakeholders en sesiones facilitadas para identificar, discutir y priorizar requisitos. Son muy efectivos para resolver conflictos y lograr consenso rápido.

- **Análisis de documentos**

Revisión de documentos existentes como manuales, formularios, reportes y flujos de proceso del negocio actual. Permite entender el contexto y descubrir requisitos implícitos.

- **Observación directa (etnografía)**

El ingeniero observa a los usuarios realizando su trabajo cotidiano para identificar necesidades que ellos mismos no saben expresar verbalmente. Es especialmente útil para descubrir requisitos implícitos.

Para concluir, un requisito de calidad debe cumplir los siguientes principios y criterios de verificación:

- **Claridad:** el requisito debe ser comprensible para cualquier lector sin necesidad de interpretación adicional. Pregunta clave:

¿Puede cualquier persona del equipo entenderlo de la misma forma?

- **Compleitud:** el requisito debe incluir toda la información necesaria: qué hace el sistema, en qué condiciones y con que restricciones. Pregunta clave:

¿Hay información que falta o que se da por supuesta?

- **Consistencia:** el requisito no debe contradecir a ningún otro requisito del mismo documento. Pregunta clave:

¿Existe algún otro requisito que entre en conflicto con este?

- **Verificabilidad:** debe ser posible demostrar mediante pruebas que el requisito ha sido implementado correctamente. Pregunta clave:

¿Cómo saber que el sistema cumple este requisito?

- **Trazabilidad:** debe poderse rastrear el origen del requisito (a quién lo solicitó y por qué) y su implementación en el código. Pregunta clave:

¿De dónde vino este requisito y cómo se implementó?

¿Cuál es la ventaja de modelar requisitos gráficamente?

- **La precisión:** un diagrama de casos de uso reduce significativamente la ambigüedad, ya que define con claridad si un actor participa o no en una interacción y si un caso de uso está o no incluido en otro. Además, estos modelos pueden analizarse de manera formal mediante herramientas automáticas que permiten detectar inconsistencias. De este modo, un requisito representado en un diagrama de clases no puede describirse de forma contradictoria en otro diagrama, garantizando la coherencia y uniformidad del modelo.

2.2. Modelos conceptuales

Son representaciones gráficas o formales que permiten describir, analizar y comunicar los requisitos de un sistema de software de forma visual y estructurada. A diferencia del lenguaje natural, que usa texto libre, los modelos conceptuales utilizan notaciones estandarizadas con símbolos, formas y conectores que tienen significados precisos y acordados por la industria. Su propósito principal es eliminar la ambigüedad del lenguaje natural, facilitar la comunicación técnica entre equipos y servir como base para el diseño e implementación del sistema.

Las características de un buen modelo conceptual son:

- **Claridad visual:** fácil de leer e interpretar por cualquier miembro del equipo.
- **Notación estándar:** usa símbolos reconocidos (UML, BPMN, etc.) para garantizar comprensión universal.
- **Trazabilidad:** cada elemento del modelo debe poder vincularse a uno o más requisitos escritos.
- **Compleitud:** representa todos los actores, procesos y relaciones relevantes del sistema.
- **Precisión:** no deja lugar a interpretaciones múltiples como puede ocurrir con el texto libre.

Los modelos conceptuales no reemplazan al lenguaje natural, lo complementan. En la práctica, el proceso de documentación sigue esta secuencia:

- a) **Captura en lenguaje natural:** se recopilan los requisitos a través de entrevistas y talleres usando lenguaje natural.
- b) **Análisis de texto:** el ingeniero identifica actores, acciones, entidades y relaciones en el texto.
- c) **Modelado:** se crean los diagramas correspondientes a partir del análisis del texto.
- d) **Validación:** se revisa con el cliente que los diagramas representan correctamente sus necesidades.
- e) **Actualización:** si el modelo cambia, el texto de requisitos también se actualiza (trazabilidad bidireccional).

Los modelos utilizan notaciones gráficas o formales para representar requisitos por medio de los siguientes diagramas:

A. Diagramas de casos de uso

Los diagramas de casos de uso son parte del estándar UML (Unified Modeling Language) y son uno de los artefactos más importantes en la ingeniería de requisitos. Muestran las interacciones entre actores externos (usuarios, sistemas externos) y el sistema que se va a desarrollar.

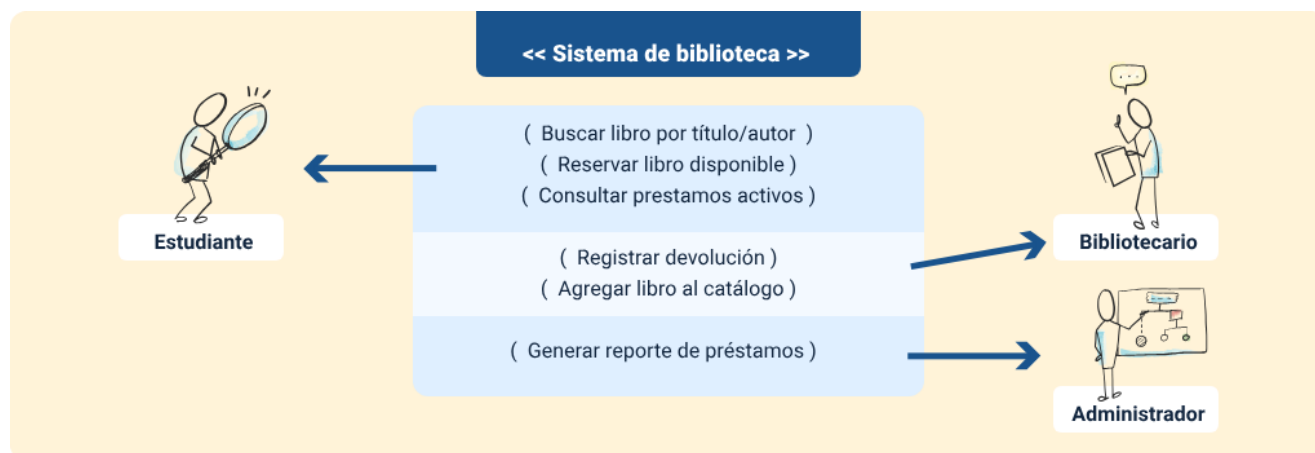
Tabla 4. Elementos principales de los diagramas de casos de uso

Elemento	Símbolo / notación	Descripción
Actor	Figura de palito (stickman)	Persona, rol o sistema externo que interactúa con el sistema. Puede ser usuario,

Elemento	Símbolo / notación	Descripción
		administrador, sistema de pagos, etc.
Caso de uso	Elipse con nombre	Acción o funcionalidad que el sistema ofrece a un actor. Describe el “qué hace el sistema”, no el “cómo lo hace”.
Asociación	Línea solida	Conecta un actor con el caso de uso que el inicia o en el que participa.
Relación <<include>>	Flecha punteada con <<include>>	Indica que un caso de uso siempre incluye el comportamiento de otro. Es obligatorio.
Relación <<extend>>	Flecha punteada con <<extend>>	Indica que un caso de uso puede extender opcionalmente el comportamiento de otro. Es condicional.
Frontera del sistema	Rectángulo que encierra los casos de uso	Delimita los límites del sistema. Los actores quedan fuera; los casos de uso, dentro.

A continuación, se relaciona un ejemplo:

Figura 1. Diagrama de casos de uso - Sistema de biblioteca



Lectura del diagrama:

- El estudiante puede buscar libros, reservarlos y consultar sus préstamos activos.
- El bibliotecario puede registrar devoluciones y agregar libros al catálogo.
- El administrador tiene acceso a los reportes de préstamos.
- La frontera del sistema delimita todo lo que el software debe implementar.

¿Como se construye un diagrama de casos de uso desde el lenguaje natural?

El proceso de extraer casos de uso a partir de un texto de requisitos sigue estos pasos:

- Identificar los SUSTANTIVOS que representan usuarios o sistemas externos
-> estos son los ACTORES.
- Identificar los VERBOS que describen acciones que el sistema debe realizar
-> estos son los CASOS DE USO.
- Determinar que actor inicia cada caso de uso y trazar la ASOCIACIÓN.

d) Identificar relaciones <<include>> (acciones siempre presentes) y <<extend>> (acciones opcionales).

e) Dibujar la FRONTERA DEL SISTEMA y organizar los elementos visualmente.

EL siguiente es un ejemplo de extracción de casos de uso

Texto en lenguaje natural:

"Los estudiantes pueden buscar libros y reservarlos si están disponibles. Para reservar, el sistema verifica primero si el estudiante tiene préstamos vencidos".

Extracción:

- Actor: estudiante.
- Caso de uso 1: buscar libro.
- Caso de uso 2: reservar libro.
- Caso de uso 3: verificar préstamos vencidos.
- Relación: "reservar libro" <<include>> "verificar préstamos vencidos".

B. Diagramas de flujo

Los diagramas de flujo (flowcharts) representan gráficamente la secuencia lógica de pasos que se siguen en un proceso o algoritmo. Son herramientas fundamentales para documentar el flujo de trabajo de un sistema antes de su implementación y para identificar decisiones, ciclos y caminos alternativos. Aunque no forman parte del estándar UML, son ampliamente utilizados en la ingeniería de requisitos por su simplicidad y claridad para representar procesos de negocio.

A continuación, se presenta una tabla que describe los símbolos estándar del diagrama de flujo:

Tabla 5. Símbolo estándar de los diagramas de flujo

Símbolo	Nombre	Uso
Óvalo / capsula	Inicio / fin (terminal)	Marca el punto de inicio o finalización del proceso. Siempre hay uno de inicio y al menos uno de fin.
Rectángulo	Proceso / acción	Representa una tarea, operación o acción que se ejecuta. Ejemplo: “registrar usuario”, “calcular total”.
Rombo / diamante	Decisión	Representa un punto de decisión con dos o más caminos posibles. Generalmente tiene salidas sí/no o verdadero/falso.
Paralelogramo	Entrada / salida	Representa datos que entran al sistema (input) o resultados que salen (output).
Flecha	Flujo de control	Indica la dirección del flujo entre los pasos del proceso.
rectángulo doble borde	Subproceso	Indica que ese paso llama a otro proceso definido en un diagrama separado.

El siguiente ejemplo de diagrama representa el flujo mediante unos pasos secuenciales:

Diagrama de flujo - Proceso de préstamo de libro

INICIO: El estudiante solicita un libro

v

[PROCESO]: Buscar libro en el catálogo del sistema

v

< DECISION > El libro está disponible?

SI v / NO --> SI: continuar | NO: notificar no disponible -> FIN

[PROCESO]: Verificar si el estudiante tiene prestamos vencidos

v

< DECISION > Tiene prestamos vencidos?

NO v / SI --> NO: continuar | SI: bloquear solicitud -> FIN

[PROCESO]: Registrar préstamo (estudiante, libro, fecha)

v

[PROCESO]: Actualizar estado del libro a “Prestado”

v

[SALIDA]: Enviar correo de confirmación al estudiante

v

FIN: Préstamo registrado exitosamente

¿Cuándo usar un diagrama de flujo?

- Para documentar el flujo de un proceso de negocio complejo con múltiples decisiones.

- Para clarificar la lógica detrás de un requisito funcional específico.
- Para detectar cuellos de botella, redundancias o pasos innecesarios en un proceso.
- Para comunicar algoritmos a desarrolladores antes de la codificación

C. Diagramas de clases

Son el artefacto central del estándar UML para representar la estructura estática de un sistema. Muestran las clases (tipos de objetos), sus atributos (datos que almacenan), sus métodos (acciones que realizan) y las relaciones entre ellas. Son la base para el diseño de la base de datos y la arquitectura orientada a objetos del sistema. Los desarrolladores usan estos diagramas directamente para implementar el código.

A continuación, se presenta una tabla con la descripción de los elementos del diagrama de clases:

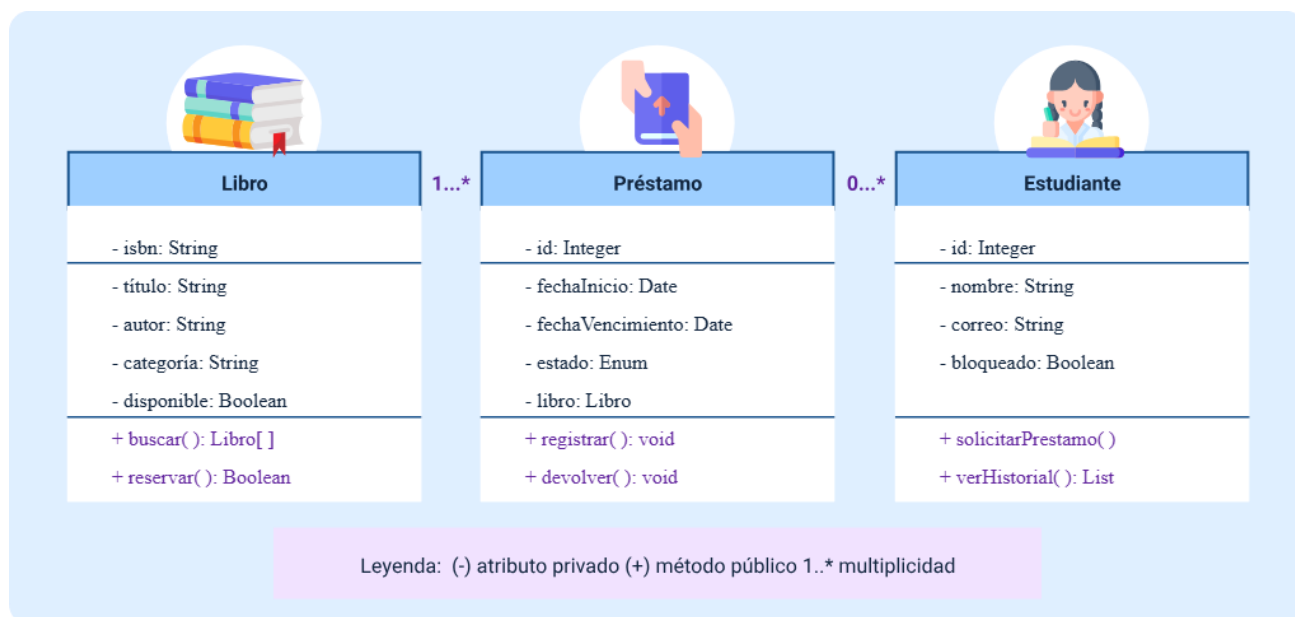
Tabla 6. Elementos de los diagramas de clase

Elemento	Descripción y notación
Clase	Rectángulo dividido en tres partes: nombre (arriba, en negrita), atributos (centro) y métodos (abajo).
Atributo	Dato que caracteriza a un objeto de la clase. Notación: visibilidad nombre: tipo. Ejemplo: - nombre: String.
Método	Acción que puede realizar un objeto. Notación: visibilidad nombre(parámetros): retorno. Ejemplo: + calcularTotal(): Double.

Elemento	Descripción y notación
Visibilidad	+ publico (accesible desde fuera), - privado (solo dentro de la clase), # protegido (herencia).
Asociación	Línea solida entre clases. Indica que una clase conoce o usa a otra.
Multiplicidad	Números en los extremos de la asociación: 1 (exactamente uno), 0..* (cero o más), 1..* (uno o más).
Composición	Rombo sólido. La clase componente no puede existir sin la clase contenedora (ciclo de vida dependiente).
Agregación	Rombo vacío. La clase componente puede existir independientemente de la clase contenedora.
Herencia	Flecha con triangulo vacío apuntando hacia la superclase. Expresa la relación “es un tipo de”.

El siguiente es un ejemplo de este tipo de diagrama:

Figura 2. Diagrama de clases - Sistema de biblioteca (simplificado)



¿Como se construye un diagrama de clases desde el lenguaje natural?

- Identificar los SUSTANTIVOS principales en el texto de requisitos -> son candidatos a CLASES.
- Para cada clase, identificar los ADJETIVOS y DATOS asociados -> son los ATRIBUTOS.
- Identificar los VERBOS asociados a cada clase -> son los MÉTODOS.
- Identificar las RELACIONES entre clases (una reserva pertenece a un estudiante y un libro).
- Definir la MULTIPLICIDAD de cada relación.
- Refinar el modelo eliminando clases redundantes o fusionando las muy similares.

Los siguientes son los tipos de datos comunes en atributos:

- **String:** texto (nombre, correo, descripción).
- **Integer:** número entero (id, cantidad, edad).
- **Double:** número decimal (precio, descuento, porcentaje).
- **Boolean:** verdadero / Falso (activo, disponible, bloqueado).
- **Date:** fecha (fechaNacimiento, fechaCreación).
- **DateTime:** fecha y hora (fechaRegistro, ultimaModificación).
- **Enum:** conjunto de valores fijos (estado: ACTIVO, INACTIVO, SUSPENDIDO).

D. Diagramas de secuencia

Son diagramas de comportamiento UML que muestran como los diferentes componentes (objetos, clases, actores o sistemas) de una aplicación interactúan entre si a lo largo del tiempo para realizar una tarea específica. El eje horizontal representa los participantes y el eje vertical representa el tiempo (fluye hacia abajo). Son especialmente útiles para documentar escenarios de casos de uso, describiendo paso a paso los mensajes que se intercambian entre componentes.

A continuación, se presenta una tabla que describe los elementos del diagrama de secuencia:

Tabla 7. Elementos de los diagramas de secuencia

Elemento	Descripción
Lifeline (Línea de vida)	Línea vertical punteada que representa la existencia de un participante (actor, objeto, sistema) durante la interacción.
Participante / actor	Rectángulo en la parte superior de la lifeline que indica el nombre del participante.
Mensaje síncrono	Flecha sólida. El emisor espera la respuesta antes de continuar. Ejemplo: llamada a método.
Mensaje de retorno	Flecha punteada. Representa la respuesta que regresa al emisor después de procesar el mensaje.
Mensaje asíncrono	Flecha con punta abierta. El emisor no espera respuesta para continuar. Ejemplo: notificación, evento.
Activación (barra)	Rectángulo delgado sobre la lifeline que indica el periodo en que el participante este activo procesando.

Elemento	Descripción
Fragmento alt	Marco rectangular con “alt” que representa condiciones alternativas (similar a if-else).
Fragmento loop	Marco rectangular con “loop” que representa una iteración (similar a un ciclo for o while).

A continuación, un ejemplo de este diagrama:

Tabla 8. Ejemplo de diagrama de secuencia - Búsqueda y préstamo de libro:

#	De	Para	Mensaje (método)	Descripción
1	Estudiante	Sistema	buscarLibro(titulo)	El estudiante ingresa el título en la interfaz.
2	Sistema	BaseDatos	consultarCatalogo(titulo)	El sistema consulta el catálogo en la base de datos.
3	BaseDatos	Sistema	retornarResultados(libros[])	La BD devuelve la lista de libros encontrados.
4	Sistema	Estudiante	mostrarResultados(libros[])	El sistema muestra los resultados al estudiante.
5	Estudiante	Sistema	seleccionarLibro(isbn)	El estudiante elige el libro que desea reservar.
6	Sistema	BaseDatos	verificarDisponibilidad(isbn)	El sistema comprueba si el libro está disponible.
7	BaseDatos	Sistema	retornarEstado(disponible=true)	La BD confirma que el libro está disponible.
8	Sistema	BaseDatos	registrarPrestamo(estudiante, isbn)	El sistema crea el registro del préstamo.

9	Sistema	Correo	enviarConfirmacion(estudiante)	El sistema solicita enviar correo de confirmación.
10	Sistema	Estudiante	mostrarConfirmacion()	El sistema confirma el préstamo al estudiante.

Interpretación del diagrama:

- Los pasos 1-4 describen el flujo de búsqueda: el estudiante busca, el sistema consulta la BD y devuelve resultados.
- Los pasos 5-7 describen la selección y verificación de disponibilidad.
- Los pasos 8-10 describen el registro del préstamo y la confirmación al usuario.
- Cada flecha representa un mensaje o llamada entre componentes, lo que ayuda a definir las interfaces del sistema.

La siguiente tabla muestra la comparación general de los cuatro modelos:

Tabla 9. Comparación modelos conceptuales

Modelo	¿Qué representa?	¿Cuándo usarlo?	Nivel técnico
Caso de uso	Interacciones entre actores y el sistema (funcionalidades).	Al inicio del proyecto para definir el alcance funcional con el cliente.	Bajo - accesible para no técnicos.
Diagrama de flujo	Secuencia de pasos y decisiones en un proceso.	Para documentar lógica de procesos complejos con muchas ramificaciones.	Bajo - medio.

Modelo	¿Qué representa?	¿Cuándo usarlo?	Nivel técnico
Diagrama de clases	Estructura del sistema: clases, atributos, métodos y relaciones.	En la fase de diseño para definir la arquitectura orientada a objetos.	Alto - orientado a desarrollo.
Diagrama de secuencia	Interacción entre componentes a lo largo del tiempo.	Para documentar el comportamiento dinámico de un caso de uso específico.	Alto - orientado a desarrollo.

- **Herramientas para crear modelos conceptuales**

Existen diversas herramientas, tanto gratuitas como de pago, que los ingenieros de requisitos usan para crear estos diagramas de forma profesional:

Tabla 10. Herramientas para crear modelos conceptuales

Herramienta	Costo / tipo	Diagramas que soporta
Draw.io (diagrams.net)	Gratuita - web y escritorio	Todos los tipos: UML, flujo, ER, redes, etc.
Lucidchart	Freemium - web colaborativa	UML, flujo, wireframes, mapas mentales.
StarUML	Freemium - escritorio	Especialización en UML: casos de uso, clases, secuencia.
Visual Paradigm	Freemium - escritorio y web	UML completo + BPMN + modelado de BD.
Enterprise Architect	Pago - profesional	UML completo, trazabilidad, generación de código.

Herramienta	Costo / tipo	Diagramas que soporta
PlantUML	Gratuita - basada en texto	UML desde código de texto plano. Ideal para automatización.
Microsoft Visio	Pago - integrado con Office	Flujo, organización, redes, UML básico.

Sin embargo, los modelos también presentan limitaciones, ya que su correcta interpretación exige que quienes los leen cuenten con formación técnica. Esto puede dificultar la comprensión por parte de clientes empresariales sin experiencia en notación UML. Además, al ser representaciones abstractas, los modelos pueden ocultar detalles importantes; por ejemplo, un diagrama de secuencia que indica que “el usuario envía datos al servidor” no especifica aspectos como la validación de datos, el manejo de errores o el comportamiento del sistema ante condiciones de red lenta.

2.3. Enfoque híbrido

Combina lo mejor de ambos mundos. Una especificación de requisitos típicamente incluye narrativa en lenguaje natural para contexto, explicación y casos de uso, complementada con diagramas que muestran relaciones complejas, flujos de datos y estructuras. Por ejemplo, podría haber un párrafo que explique el concepto de un cliente registrado en un sistema de comercio electrónico, seguido de un diagrama que muestre cómo se relacionan los clientes con sus órdenes, direcciones de envío y métodos de pago.

En la práctica, los proyectos de software raramente usan un solo tipo de documentación. Los modelos híbridos combinan el lenguaje natural con uno o varios modelos conceptuales (diagramas UML, flujos, tablas) dentro de un mismo documento de requisitos, aprovechando las fortalezas de cada enfoque y compensando sus debilidades.

El objetivo es lograr documentación que sea a la vez comprensible para el cliente y precisa para el equipo técnico, sin duplicar el esfuerzo ni generar inconsistencias.

La siguiente tabla relaciona las combinaciones más usadas:

Tabla 11. Combinaciones híbridas

Combinación híbrida	¿Cuándo se usa?	Ejemplo de aplicación
Lenguaje natural + casos de uso	Para presentar el alcance del sistema a clientes. El texto describe el contexto y el diagrama muestra el alcance funcional.	Una tabla de casos de uso con su descripción en texto adjunta al diagrama.
Lenguaje natural + diagrama de flujo	Para documentar reglas de negocio complejas. El texto describe la regla y el flujo muestra los caminos posibles.	Un requisito de validación de pago descrito en texto + el flujo de decisión del proceso.
Lenguaje natural + diagrama de clases	Para definir el modelo de datos. El texto describe las entidades y el diagrama muestra sus relaciones y multiplicidades.	Especificación de entidades de un e-commerce: texto + clases producto, orden, cliente.
Lenguaje natural + diagrama de secuencia	Para documentar integraciones entre sistemas. El texto describe el escenario y la secuencia muestra los mensajes intercambiados.	Integración con pasarela de pago: texto del escenario + secuencia de llamadas API.

Combinación híbrida	¿Cuándo se usa?	Ejemplo de aplicación
Historias de usuario + criterios de aceptación + flujo	En metodologías ágiles. La historia describe el valor, los criterios definen la verificación y el flujo muestra el proceso.	Historia: “como comprador desea pagar con tarjeta”. Criterios medibles + flujo del checkout.

La selección entre estos enfoques depende del contexto del sistema. En proyectos pequeños y relativamente simples, el uso de lenguaje natural puede ser suficiente. En cambio, en sistemas complejos, con múltiples actores e interacciones, los modelos resultan fundamentales. Por esta razón, la mayoría de las organizaciones profesionales adopta un enfoque híbrido, ya que reconoce que las distintas audiencias tienen necesidades diferentes: los clientes comprenden mejor la narrativa, los arquitectos requieren los modelos y los desarrolladores se benefician de ambos enfoques.

Para finalizar esta temática de tipos de documentación de requisitos, es importante analizar la siguiente tabla:

Tabla 12. Comparación de enfoques de documentación de requisitos

Enfoque	Ventajas	Limitaciones
Lenguaje natural	Accesible a todas las audiencias, excelente para narrativa y contexto, requiere mínima capacitación, flexible en expresión y permite descripción de requisitos complejos.	Ambiguo, sujeto a múltiples interpretaciones, difícil de validar formalmente, puede ser verboso, inconsistencias y no siempre detectadas.
Modelos conceptuales (UML, diagramas)	Precisión, captura relaciones complejas visualmente, permite análisis formal automático,	Requiere capacitación técnica, puede ser abstracto, no todas las personas técnicas conocen UML, tiempo de aprendizaje y

Enfoque	Ventajas	Limitaciones
	consistencia entre vistas y facilita detección de inconsistencias.	puede ocultar detalles importantes.
Híbrido (narrativa + modelos)	Combina claridad de narrativa con precisión de modelos, múltiples perspectivas, flexible, comprensible por diferentes audiencias y profesional.	Mayor esfuerzo de documentación, necesita coordinación cuidadosa entre textos y diagramas, requiere competencia en múltiples técnicas.

3. Buenas prácticas de documentación

La documentación de requisitos no es solo sobre qué documentar, sino cómo hacerlo de manera profesional, consistente y clara. Esto es donde entran en juego los estándares y las mejores prácticas. En Colombia, el estándar NTC-1486 del ICONTEC (Instituto Colombiano de Normas Técnicas y Certificación) establece directrices para la presentación de trabajos escritos. Aunque originalmente fue diseñado para tesis académicas, muchas de sus recomendaciones —estructura clara, márgenes consistentes, tipografía legible— son perfectamente aplicables a documentos técnicos de requisitos.

Por su parte, el estándar APA (American Psychological Association) es ampliamente utilizado para citas y referencias. Cuando se documenta un requisito, es importante indicar su origen: ¿proviene de una regulación gubernamental específica?, ¿de una solicitud particular del cliente?, ¿de un estándar de industria?, ¿de un análisis interno? Las citas correctas permiten rastreabilidad y facilitan argumentación si surge una disputa sobre por qué se incluyó un requisito particular.

Los estándares y formatos son solo la estructura, lo verdaderamente importante es la calidad de cómo se escriben los requisitos individuales. Aquí hay siete principios fundamentales para escribir requisitos de calidad:

- **Claridad**

Un requisito debe ser expresado en lenguaje claro, preciso y sin ambigüedades. Considere la diferencia entre: "el sistema debe ser eficiente" (ambiguo, no verificable) versus "el sistema debe procesar un lote de 1000 órdenes de compra en menos de 5 minutos cuando se ejecuta en un servidor con procesador Intel Xeon E5-2690 y 32GB

de RAM" (específico, verificable, cuantificable). La claridad también significa evitar jerga innecesaria. Si está documentando un requisito para ser leído por stakeholders no técnicos, no se debe asumir que se entiendan términos como "API REST" o "microservicios" sin explicación.

- **Compleitud**

Un requisito debe contener toda la información necesaria para ser comprendido e implementado sin requerir información adicional de otras fuentes. Si se dice "el sistema debe generar reportes mensuales", se ha dejado muchas preguntas sin responder: ¿qué información contienen los reportes?, ¿qué formato tienen (PDF, Excel, HTML)?, ¿a quién se envían?, ¿cuándo se generan exactamente el primer día del mes, el último día, la última semana?, ¿se incluyen gráficos? Un requisito completo respondería todas estas preguntas o las dejaría claramente para ser discutidas después.

- **Consistencia**

La terminología utilizada debe ser uniforme en todo el documento. Si se hace referencia a un "usuario" en un lugar, "cliente" en otro y "persona" en un tercero, se está creando confusión innecesaria. Aunque el significado puede ser evidente en contexto, la inconsistencia aumenta la posibilidad de malentendidos. Las organizaciones profesionales frecuentemente mantienen un glosario de términos como parte de su especificación de requisitos.

- **Trazabilidad**

Cada requisito debe tener un identificador único, como REQ-001, REQ-002, etc. Este identificador permite referenciar el requisito en otras partes del documento, en planes de prueba, en cambios solicitados, en discusiones con el cliente. Sin trazabilidad,

es fácil perder referencias, olvidar validar un requisito o implementar una funcionalidad sin haberse dado cuenta que fue solicitada.

- **Viabilidad**

Los requisitos deben ser realistas y alcanzables con la tecnología y los recursos disponibles. Un requisito imposible de implementar genera frustración, aumenta costos y puede llevar a fallos de proyectos. Si el presupuesto es de \$100.000 y alguien requiere que el sistema use inteligencia artificial de última generación que costaría \$500.000, no es un requisito viable. Es responsabilidad del ingeniero de requisitos ayudar al cliente a entender las limitaciones y a hacer compensaciones inteligentes.

- **Verificabilidad**

Debe ser posible verificar de manera objetiva que un requisito ha sido implementado correctamente. Requisitos como “debe ser fácil de usar” no son verificables, ya que el término “fácil” es ambiguo. En cambio, un requisito como “el 90 % de los usuarios sin capacitación debe poder completar la tarea de registrar una nueva orden en menos de tres minutos” sí es verificable. Para ello, pueden evaluarse usuarios reales, cronometrar la tarea y medir objetivamente si el requisito se cumple.

- **Independencia**

Los requisitos deben ser tan independientes como sea posible. No debe ser necesario entender un requisito para comprender otro. Si el requisito A se define como "implementar la funcionalidad descrita en el requisito B", entonces A no es realmente independiente. Requisitos interdependientes son más difíciles de priorizar, de cambiar y de asignar a diferentes desarrolladores.

Ahora, se relacionan algunos ejemplos de requisitos mal escritos versus bien escritos:

- **Ejemplo 1**

Mal escrito: "el sistema debe tener una buena interfaz de usuario".

Por qué está mal: ¿qué significa "buena"? No es verificable, no es específico.

Bien escrito: "la interfaz de usuario debe usar colores de alto contraste (diferencia de luminancia de al menos 4.5:1 según estándar WCAG 2.1 AA) para legibilidad en condiciones de iluminación variable. Todos los botones de acción principal deben ser de al menos 44x44 píxeles para accesibilidad táctil. Los tiempos de respuesta de la interfaz a acciones del usuario no deben exceder 200 milisegundos para mantener la sensación de reactividad inmediata".

- **Ejemplo 2**

Mal escrito: "el sistema debe procesar transacciones rápidamente".

Por qué está mal: ¿qué tan rápido es "rápidamente"? ¿en relación a qué baseline?

“Un baseline es un valor o estándar con el que se comparan los resultados”.

Bien escrito: "el sistema debe procesar transacciones estándar de compra en menos de 2 segundos para el 95 % de los casos bajo carga normal (hasta 1000 transacciones por minuto) y en menos de 5 segundos para el 100 % de los casos incluso bajo carga pico (hasta 5000 transacciones por minuto)".

“El sistema debe procesar transacciones rápidamente”

El problema es que “rápidamente” no tiene un baseline; es decir, no hay un punto de referencia claro para medir qué tan rápido es.

- **Ejemplo 3**

Mal escrito: "el sistema debe ser seguro"

Por qué está mal: "seguro" es demasiado ambiguo. ¿Seguro contra qué?

Bien escrito: "el sistema debe encriptar todos los datos en tránsito usando TLS 1.3 o superior. Las contraseñas de usuario deben almacenarse usando el algoritmo bcrypt con un factor de costo de al menos 12. El acceso a datos sensibles debe requerir autenticación multifactor (contraseña más código de autenticación temporal). Los intentos de acceso fallidos deben ser limitados a máximo 5 intentos antes de bloquear la cuenta temporalmente por 15 minutos."

Estas son las bases de una documentación de requisitos de calidad. Sin embargo, existe un nivel adicional, que consiste en comprender el porqué de los requisitos. El cliente rara vez expresa directamente: “se requiere un sistema que maneje 1.000 transacciones por minuto”. Por lo general, lo que manifiesta es un problema o una necesidad, como: “se busca mejorar la experiencia de los clientes en línea durante las épocas de ventas”.

Por ello, el ingeniero de requisitos debe ser capaz de traducir esa necesidad en requisitos específicos y formular preguntas ejemplo como:

- ¿Por qué 1000 transacciones?

- ¿Por qué ese es el pico esperado durante Black Friday?
- ¿Cuál es la consecuencia si no se alcanza?
- ¿Hay clientes frustrados?
- ¿Existe pérdida de ventas?
- ¿Hay daño a la marca?

Entender el porqué detrás de cada requisito permite tomar compensaciones inteligentes si es necesario, priorizar correctamente y comunicar la importancia relativa de diferentes requisitos a los desarrolladores.

4. Informe de requisitos

Cuando una organización decide construir un nuevo sistema de software de cierta complejidad, es fundamental elaborar una especificación de requisitos (SRS – Software Requirements Specification), también conocida como informe de requisitos. Este artefacto va mucho más allá de un registro administrativo: funciona como un contrato entre el cliente, que financia el proyecto y el equipo de desarrollo, que lo construye. Además, sirve como referencia para la arquitectura, el diseño y las decisiones de implementación, y constituye la base sobre la cual se evaluará la aceptación del producto final.

- **Estándar IEEE**

El estándar IEEE 830, actualmente reemplazado por IEEE 29148, proporciona una estructura de referencia para la elaboración de especificaciones de requisitos. Este estándar ha evolucionado a partir de décadas de experiencia en la industria del software y refleja el consenso de expertos sobre los elementos esenciales que debe contener una especificación de requisitos profesional. Aunque no es obligatorio seguir el estándar de manera estricta, muchas organizaciones adaptan su estructura a necesidades específicas; sin embargo, resulta fundamental comprender los principios que lo sustentan.

La siguiente imagen clarifica un poco esta actualización:

Figura 3. Actualización estándar IEEE



Evolución del estándar de requisitos de software

IEEE 830 - Software Requirements Specification (SRS)

- Requisitos funcionales
- Requisitos no funcionales

Actualización del estándar

IEEE 29148 – Requirements Engineering Standard

Stakeholders - Diagramas de sistemas – Validación - Verificación

El IEEE 29148 reemplaza y amplía el IEEE 830 integrando mejores prácticas modernas de ingeniería de requisitos.

- **IREB**

El International Requirements Engineering Board (IREB) es una organización profesional dedicada a la certificación de ingenieros de requisitos. Para obtener una

certificación IREB, el profesional debe demostrar un conocimiento sólido de los principios de la ingeniería de requisitos. Entre las áreas evaluadas se incluye el dominio de estándares como IEEE 830 y las buenas prácticas en documentación. IREB define un body of knowledge que establece las competencias esenciales de un especialista en requisitos.

Una especificación de requisitos típicamente tiene estas secciones principales:

- **Introductoria**

Establece el contexto, describe el propósito del documento, define el alcance del proyecto y proporciona referencias a documentos relacionados.

- **Descripción general del producto**

Explica desde una perspectiva de alto nivel qué va a hacer el sistema, quiénes son los usuarios y cuáles son las restricciones principales.

- **Requisitos específicos**

Detalla cada requisito funcional, no funcional, de interfaz y de restricción.

Cada requisito de software debe contar con una identificación única, una descripción clara y verificable, indicar su origen, establecer su prioridad y registrar cualquier observación relevante. Esta información permite comprender no solo qué debe hacer el sistema, sino también por qué se requiere y qué tan crítico resulta para el negocio. Para asegurar que estos requisitos no se pierdan a lo largo del proyecto, se emplea la matriz de trazabilidad, la cual vincula cada requisito con otros artefactos clave, como los casos de prueba, los componentes de diseño y, cuando aplica, los requisitos de nivel superior. Como complemento, la especificación puede incluir

apéndices con diagramas (casos de uso, flujos de datos o estados), prototipos, modelos de datos, un glosario de términos y análisis de riesgos, los cuales amplían y refuerzan la comprensión del sistema.

En este contexto, un documento de especificación de requisitos para un sistema de mediana complejidad suele tener entre 50 y 200 páginas, ya que debe cubrir de forma detallada las necesidades del sistema y sus restricciones. En proyectos de mayor envergadura, este volumen puede aumentar considerablemente. Sin embargo, este documento no está diseñado para ser leído de principio a fin por todos los interesados, sino para ser consultado según el rol que cada actor desempeña. Un directivo, por ejemplo, suele enfocarse en la introducción y la visión general; un desarrollador revisa los requisitos asociados al módulo que implementa; y el equipo de aseguramiento de la calidad se concentra en los requisitos y en la matriz de trazabilidad para verificar que todas las funcionalidades sean correctamente probadas.

Precisamente, uno de los elementos más críticos —y con frecuencia subestimado— dentro de la especificación de requisitos es la matriz de trazabilidad. Esta se presenta como una tabla que muestra cómo cada requisito se relaciona con los diferentes artefactos del proyecto. En ella, cada fila corresponde a un requisito y las columnas indican qué caso de prueba lo valida, qué componente de diseño lo implementa o qué historia de usuario lo describe. Gracias a esta relación explícita, si durante la fase de pruebas se detecta que un requisito no está siendo validado, la matriz lo evidencia de inmediato. De igual forma, si en el desarrollo se identifica que un requisito no fue implementado, la trazabilidad permite detectarlo oportunamente.

En consecuencia, una matriz de trazabilidad bien construida se convierte en un mecanismo de control de calidad fundamental, ya que previene la omisión,

implementación parcial o validación incompleta de los requisitos. Más allá de ser un simple registro, actúa como un enlace permanente entre requisitos, diseño, desarrollo y pruebas, fortaleciendo la calidad del software y reduciendo riesgos a lo largo de todo el ciclo de vida del proyecto.

Complementando lo anterior, se relaciona un ejemplo de cómo podría verse una matriz de trazabilidad simplificada:

Tabla 13. Ejemplo matriz de requisitos

Requisito ID	Descripción breve	Casos de prueba	Componentes de diseño
REQ-001	Usuario puede registrarse con email y contraseña.	TC-001, TC-002.	Módulos: Autenticación, Usuario.
REQ-002	Sistema valida fortaleza de contraseña (mínimo 12 caracteres).	TC-003, TC-004.	Módulos: Validación, Módulo Seguridad.
REQ-003	Sistema envía email de confirmación en menos de 30 segundos.	TC-005.	Módulos: Email, Cola de Tareas.
REQ-004	Usuario puede recuperar contraseña olvidada.	TC-006, TC-007, TC-008.	Módulos: Recuperación, Email.

Esta matriz, aunque simplificada, muestra inmediatamente qué se está probando y qué módulos están involucrados en cada requisito. Cuando un requisito cambia, la matriz ayuda a identificar qué casos de prueba necesitan actualizarse y qué módulos necesitarán cambios de diseño.

5. Historias de usuario

Para conocer un poco de historia, se sugiere revisar la siguiente línea de tiempo:

- Décadas de 1980–1990. Predominio de metodologías tradicionales como e, con especificaciones extensas y detalladas.
- Finales de los años 1990. Surgen cuestionamientos sobre la rigidez de las metodologías tradicionales y el alto costo de la documentación extensa, lo que da origen a Extreme Programming (XP), con un enfoque iterativo y centrado en el cliente.
- Inicios de los 2000. Se consolidan Scrum y otras metodologías ágiles.
- Enfoque ágil. Aparecen las historias de usuario como una forma simple y centrada en el usuario para expresar requisitos.

Como resultado del surgimiento de las metodologías ágiles a finales de los años noventa, se consolidó una forma más simple y centrada en el usuario para expresar los requisitos: las historias de usuario. Estas se redactan en lenguaje natural y describen necesidades reales desde la perspectiva de quien utiliza el sistema.

El formato más utilizado se estructura así:

“Como [tipo de usuario], se requiere [funcionalidad], con el fin de [beneficio o valor]”.

Algunos ejemplos prácticos son:

- Como cliente de comercio electrónico, se requiere visualizar recomendaciones personalizadas en la página de inicio, con el fin de facilitar la identificación rápida de productos de interés.

- Como administrador del sistema, se requiere la capacidad de deshabilitar cuentas de usuario sin eliminar datos históricos, con el fin de conservar registros completos de usuarios inactivos.
- Como usuario de una aplicación móvil, se requiere que el sistema funcione adecuadamente con conexiones lentas, con el fin de permitir el acceso en condiciones de señal limitada.

¿Qué hace que las historias de usuario sean poderosas?

- A. El foco en el usuario:** no es una lista de características técnicas, es una expresión de valor desde la perspectiva de quien va a usar el sistema. El beneficio final es claro.
- B. El tamaño pequeño:** una historia de usuario típicamente representa una cantidad de trabajo que puede ser completada en una semana de desarrollo por un pequeño equipo. Esto hace que sean manejables, estimables y reducen el riesgo de estimaciones incorrectas.
- C. Las historias de usuario enfatizan la conversación:** el texto escrito es deliberadamente incompleto. Es un recordatorio para que los desarrolladores y el cliente tengan una conversación. "Como usuario desea buscar productos" es incompleto. ¿Por qué campos?, ¿búsqueda de texto libre o búsqueda facetada?, ¿autocompletación?, ¿corrección de errores tipográficos? Esas conversaciones surgen durante el desarrollo, lo que permite que el cliente refine y aclare sus necesidades exactas basado en la retroalimentación visual.
- D. Las historias de usuario son ideales para trabajo iterativo:** un proyecto ágil, no se debe intentar completar todo el sistema, pues cada iteración

(típicamente de 1-4 semanas) entrega un conjunto de historias de usuario completadas. El cliente ve progreso constantemente, las historias completadas se cierran y se hacen más historias y finalmente, el proyecto avanza gradualmente, con ajustes basados en lo que se aprende.

Pero aquí está lo crítico: las historias de usuario no reemplazan las especificaciones formales. Esto es un error común en organizaciones que adoptan ágil por primera vez. Si se está construyendo un sistema crítico para una institución financiera o para un dispositivo médico regulado, todavía se necesitan especificaciones formales. Las historias de usuario pueden ser excelentes para gestionar el trabajo del desarrollo iterativo, pero necesita documentación formal para auditoría, cumplimiento regulatorio y referencia futura.

La mejor práctica es combinar ambos enfoques: historias de usuario para gestionar el trabajo iterativo y comunicación con el cliente durante el desarrollo, y una especificación formal que se produce gradualmente a medida que se refinan las historias. Cuando una historia se entiende completamente, se documenta formalmente en la especificación de requisitos de software.

Ahora, dentro de cada historia de usuario, están los criterios de aceptación. Estos son las condiciones específicas que deben cumplirse para que la historia se considere completa; son el puente entre la narrativa de la historia ("como cliente desea buscar productos") y los detalles técnicos específicos ("debe soportar búsqueda por texto libre, debe manejar hasta 10,000 productos, debe retornar resultados en menos de 1 segundo").

A continuación, se relaciona un ejemplo de historia de usuario con criterios de aceptación, partiendo de lo siguiente:

“Como gerente de ventas, desea generar reportes de ventas por región, para que pueda identificar dónde está el desempeño más fuerte”.

- Criterios de aceptación
 - El gerente puede seleccionar un rango de fechas (mes/año) de un calendario visual.
 - El gerente puede seleccionar una o múltiples regiones de una lista desplegable.
 - El sistema genera un reporte con las siguientes columnas: Región, Ventas Totales, Número de Transacciones, Promedio de Venta, Variación Respecto al Mes Anterior.
 - Los números se pueden exportar a formato Excel.
 - El reporte se genera en menos de 5 segundos, incluso si cubre 12 meses de datos.
 - El reporte incluye un gráfico de barras mostrando comparación entre regiones.
 - El gerente puede guardar sus configuraciones de reporte favoritas y acceder a ellas rápidamente.

- Notas técnicas
- Datos extraídos del almacén de datos, no de base de datos operacional (para no afectar desempeño).
- Requiere agregación pre-calculada nightly.
- Acceso requiere rol "Gerente" o superior.
- Una historia bien escrita tiene la característica INVEST, un acrónimo que significa:
 - Independent (independiente): no depende de otras historias para ser comprendida.
 - Negotiable (negociable): los detalles no están completamente definidos, hay espacio para conversación.
 - Valuable (valiosa): proporciona valor claro a los usuarios finales.
 - Estimable (estimable): el equipo puede estimar razonablemente el esfuerzo requerido.
 - Small (pequeña): puede ser completada en una iteración (típicamente 1-4 semanas).
 - Testable (testeable): hay criterios claros para verificar que se completó.

Si una historia no cumple con estos criterios, probablemente necesita ser refinada o dividida en historias más pequeñas.

6. Listas de chequeo para validación de información

A lo largo de este componente se ha aprendido qué caracteriza a un buen requisito: claridad, completitud, trazabilidad, verificabilidad, entre otros criterios. Sin embargo, surge una pregunta clave: ¿cómo puede una organización asegurarse de que todos los requisitos cumplan de manera consistente con estos estándares? Una herramienta sencilla pero muy efectiva para lograrlo es la lista de chequeo.

Una lista de chequeo de requisitos consiste en un conjunto de preguntas que se aplican a cada requisito para verificar su calidad frente a criterios previamente definidos. Su principal fortaleza es la simplicidad, ya que no requiere herramientas sofisticadas ni procesos complejos. El revisor recorre cada pregunta, marca “sí” o “no” y cuando identifica un “no”, sabe que el requisito necesita ser ajustado o refinado.

Las organizaciones con mayor madurez en ingeniería de requisitos suelen desarrollar listas de chequeo propias, construidas a partir de su experiencia. Estas pueden variar según el tipo de proyecto; por ejemplo, un sistema de software médico exige criterios distintos a los de una aplicación de entretenimiento, debido a los diferentes niveles de riesgo y calidad esperados. Aun así, existen principios generales que son comunes y aplicables a cualquier lista de chequeo de requisitos.

La siguiente es una lista de chequeo genérica que puede ser usada para revisar requisitos individuales de calidad de software:

Tabla 14. Lista de chequeo sobre requisitos de calidad de software

Pregunta de evaluación	Respuesta (Sí/No)	Observaciones
1. ¿El requisito tiene un identificador único (REQ-001, etc.)?		
2. ¿El lenguaje es claro y específico, sin términos vagos o ambiguos?		
3. ¿El requisito contiene toda la información necesaria para implementarse?		
4. ¿Se puede verificar objetivamente que el requisito fue implementado?		
5. ¿Es viable implementarlo con la tecnología y recursos disponibles?		
6. ¿Puede trazarse a casos de prueba, diseño o historias de usuario?		
7. ¿Es consistente con otros requisitos del sistema?		
8. ¿Se conoce el origen del requisito (cliente, regulación, estándar)?		
9. ¿Tiene una prioridad definida (crítico, alto, medio, bajo)?		
10. ¿No duplica otro requisito existente?		
11. ¿Es realmente necesario para el sistema o negocio?		
12. ¿Aporta valor claro al usuario o al negocio?		
Puntaje de calidad del requisito	0	

Pregunta de evaluación	Respuesta (Sí/No)	Observaciones
Porcentaje de cumplimiento	0	

Cada punto de la anterior lista de chequeo tiene una representación y significado específico que es:

- a) Identificación única
- b) Claridad
- c) Completitud
- d) Verificabilidad
- e) Viabilidad
- f) Trazabilidad
- g) Consistencia
- h) Origen
- i) Prioridad
- j) No redundancia
- k) Necesidad
- l) Valor

Recuerde: si un requisito falla en cualquiera de estos puntos, debería ser devuelto al remitente para refinamiento antes de ser aceptado en la especificación.

Ahora, más allá de revisiones individuales, también hay listas de chequeo para la especificación completa de requisitos con los siguientes ítems:

- ¿Se ha completado la sección de introducción con propósito, alcance, e información general?
- ¿Se ha incluido una descripción clara de los usuarios finales y sus necesidades?
- ¿Se han identificado todos los stakeholders relevantes?
- ¿Se han documentado todas las restricciones y limitaciones?
- ¿Hay una distinción clara entre requisitos funcionales y no funcionales?
- ¿Se incluye una matriz de trazabilidad completa?
- ¿Hay apéndices con diagramas que clarifiquen interacciones complejas?
- ¿Se ha revisado la especificación por lo menos dos personas independientes?
- ¿El cliente ha revisado y aprobado la especificación?
- ¿La especificación es editable y versionable en un sistema de control de cambios?

Estas listas de chequeo pueden parecer simples, pero aplicarlas consistentemente mejora significativamente la calidad de los requisitos y reduce problemas más adelante en el ciclo de vida del proyecto.

7. Técnicas para validar requisitos

Hay una distinción importante en ingeniería de requisitos entre verificación y validación, aunque los términos a menudo se confunden:

- **Verificación**

Responde a la pregunta: "¿se construye el producto correctamente?" Es decir, ¿se implementan los requisitos especificados?

- **Validación**

Responde a la pregunta: "¿se construye el producto correcto?" Es decir, ¿son los requisitos que especifican realmente lo que el cliente necesita?

Ambas son críticas, pero validación es donde frecuentemente se hace menos énfasis y es donde más sorpresas negativas ocurren. Un equipo puede construir perfectamente lo que fue especificado, cumplir 100 % de los requisitos y, aun así, entregar un producto que no satisface al cliente porque los requisitos mismos fueron incorrectos, incompletos o no alineados con las necesidades reales.

¿Cómo valida un equipo que los requisitos son correctos? Existen múltiples técnicas, cada una con fortalezas y limitaciones:

- **Inspección formal**

Una inspección formal es una revisión estructurada donde un equipo multidisciplinario examina los requisitos bajo criterios predefinidos. El equipo típicamente incluye: un inspector (moderador), un lector (presenta los requisitos), un escriba (documenta defectos), un cliente o representante del usuario (verifica contra necesidades), un desarrollador (evalúa implementabilidad), y posiblemente otros roles.

El proceso es sistemático y se desarrolla de la siguiente manera: el lector presenta un requisito o un pequeño grupo de requisitos. El equipo lo examina contra una lista de chequeo predefinida, buscando defectos: ¿es ambiguo?, ¿es incompleto?, ¿contradice otro requisito?, ¿es irrealista? Los defectos se documentan sin juzgar. No es un evento de culpa—es un evento de mejora de calidad. Una inspección formal bien ejecutada puede identificar entre el 70 % y el 80 % de los defectos en los requisitos, y hacerlo en etapas tempranas del proyecto, cuando las correcciones resultan considerablemente menos costosas.

- **Revisión guiada**

Una revisión guiada es menos formal que una inspección. En ella un facilitador guía al grupo a través de los requisitos, típicamente usando escenarios o casos de prueba como guía. Por ejemplo, se podría decir: "imaginen que es un viernes a las 5 PM, hay 5000 usuarios conectados al sistema y ocurre un pico de transacciones: ¿qué dice el requisito de rendimiento sobre esto?, ¿qué pasaría en realidad?". Este tipo de pensamiento de escenarios frecuentemente revela defectos que no serían evidentes leyendo el texto puro.

Las revisiones guiadas son menos costosas en tiempo que las inspecciones formales, pero también menos sistemáticas. Pueden saltarse temas si el facilitador no es diligente, pero el componente de escenarios las hace particularmente efectivas para descubrir requisitos faltantes o requisitos que conflictúan bajo circunstancias particulares.

- **Opinión de expertos**

Un experto en el dominio o en la tecnología relevante revisa los requisitos y proporciona retroalimentación. Por ejemplo, si los requisitos especifican que el sistema debe usar inteligencia artificial para predicción, un experto en machine learning podría revisar y decir: "los datos disponibles no son suficientes para entrenar un modelo con la precisión requerida" o "el algoritmo especificado no es el más apropiado para este uso".

La opinión de expertos es rápida y puede identificar problemas fundamentales de viabilidad; pero es subjetiva y dependiente de la disponibilidad y la disposición del experto. También está sujeta a sesgos personales.

- **Prototipado y demostraciones**

En lugar de simplemente leerse los requisitos, los usuarios finales ven o interactúan con prototipos del sistema; esto genera retroalimentación práctica. Un usuario puede ver un prototipo de interfaz de usuario y darse cuenta inmediatamente de lo siguiente: "espere, no necesito poder hacer eso, pero necesito poder hacer esto otro que no está aquí". La validación a través de prototipos es especialmente efectiva para requisitos de interfaz de usuario y requisitos sobre flujos de trabajo complejos.

¿Cuál técnica debe una organización usar?

Las mejores prácticas involucran combinar múltiples técnicas, comenzando con una lista de chequeo inicial, continuando con una revisión guiada, utilizando prototipos para la validación de la interfaz de usuario y buscando la opinión de expertos en áreas de riesgo o tecnologías nuevas; esta combinación crea múltiples capas de validación

que aumentan la probabilidad de que los requisitos representen realmente las necesidades del cliente.

8. Versionamiento de requisitos: gestión de cambios

Uno de los aspectos más complejos de la ingeniería de requisitos es que estos no son estáticos. Un requisito que fue cuidadosamente redactado, revisado, validado y aprobado en una fase inicial puede recibir solicitudes de cambio pocas semanas después. Esto no representa un fallo del proceso, sino una característica natural del desarrollo de software. A medida que el cliente observa el producto en construcción, adquiere mayor comprensión de sus necesidades; adicionalmente, los cambios del mercado, la aparición de nueva información y el descubrimiento de limitaciones técnicas hacen necesario ajustar lo definido inicialmente.

Por esta razón, el desafío no consiste en evitar los cambios de requisitos —lo cual no es posible—, sino en gestionarlos de manera controlada para que el proyecto no pierda estabilidad.

La gestión de cambios de requisitos se apoya en un proceso formal que regula cómo estos evolucionan a lo largo del proyecto. Dicho proceso suele incluir el registro de una solicitud de cambio (Change Request – CR), el análisis de su impacto, la decisión de aprobarla, rechazarla o aplazarla, su implementación en caso de aprobación y el cierre formal de la solicitud.

Cuando se presenta una solicitud de cambio, esta debe documentar claramente qué se solicita, por qué se requiere, quién la propone y en qué plazo se necesita. Idealmente, esta información debe registrarse por escrito y no limitarse a acuerdos verbales, ya que la informalidad suele generar malentendidos y confusión.

Una vez documentada, la solicitud es evaluada por un equipo responsable, que analiza qué requisitos se ven afectados, qué componentes de diseño o código deben modificarse, cuál es el esfuerzo estimado, el impacto en el cronograma y los riesgos asociados.

Con base en este análisis, se toma una decisión debidamente documentada. La solicitud puede aprobarse, rechazarse o postergarse para una fase posterior. En algunos casos, por ejemplo, un cambio puede implicar un incremento significativo del presupuesto con un beneficio limitado, lo que justifica aplazarlo hasta que el cliente confirme su disposición a asumir el costo adicional.

Si la solicitud es aprobada, los requisitos se actualizan en la especificación correspondiente. Cuando el cambio es relevante, se incrementa la versión del documento y se registra un historial que indique qué se modificó, cuándo ocurrió y quién autorizó el cambio. Esta información se comunica a todos los interesados del proyecto —desarrolladores, arquitectos y personal de pruebas— para asegurar la correcta alineación del trabajo.

Finalmente, mantener un registro completo de todas las solicitudes de cambio, tanto aprobadas como rechazadas, es una buena práctica. Este historial proporciona trazabilidad sobre la evolución del sistema y permite entender, incluso años después, las decisiones técnicas y funcionales tomadas durante el desarrollo.

9. Herramientas y tecnologías para la gestión de requisitos

En los primeros años de la ingeniería de requisitos, estos se documentaban principalmente en archivos de texto como Word o PDF. El control de versiones se realizaba de forma manual mediante múltiples copias del mismo archivo ("Especificación_v1.0.docx", "Especificación_v1.1_FINAL.docx", "Especificación_v1.1_FINAL_REVISADO.docx", "Especificación_v1.1_FINAL_REVISADO_2.docx") y la trazabilidad se gestionaba con tablas o referencias cruzadas. Los cambios solían registrarse a través de comentarios de revisión, lo que funcionaba únicamente en proyectos muy pequeños.

Este enfoque se vuelve rápidamente insostenible en proyectos de mayor complejidad. La edición simultánea de documentos, la pérdida de comentarios, la falta de un control real de versiones y la desactualización de matrices de trazabilidad o casos de prueba generan errores, retrabajo y una pérdida progresiva del control sobre los requisitos.

Para superar estas limitaciones surgieron herramientas especializadas de gestión de requisitos. Una de las soluciones pioneras fue IBM Rational DOORS, desarrollada por IBM, que introdujo el concepto de un repositorio centralizado de requisitos. En DOORS, cada requisito cuenta con un identificador único, metadatos como origen, prioridad y estado, y vínculos con otros requisitos, casos de prueba y componentes de diseño. La trazabilidad se mantiene de forma automática, de modo que cualquier cambio en un requisito se refleja en los artefactos relacionados.

No obstante, DOORS suele implicar altos costos de licenciamiento, que pueden alcanzar cifras elevadas para una organización. Por esta razón, muchas empresas optan

por alternativas más accesibles, incluidas herramientas de código abierto y plataformas modernas ampliamente adoptadas en la industria.

Entre estas alternativas se encuentran:

- **Jira (Atlassian)**

Originalmente un sistema de seguimiento de defectos, Jira evolucionó para también manejar requisitos. Muchos equipos usan Jira issues para requisitos/historias de usuario y vinculan a casos de prueba en herramientas de prueba.

- **Azure DevOps (Microsoft)**

Suite integrada que incluye gestión de requisitos (epics, características, user stories), gestión de cambios, seguimiento de defectos y análisis.

- **Traceability Matrix Tools**

Herramientas especializadas enfocadas específicamente en matrices de trazabilidad y control de cambios.

- **Specification by Example Tools**

Herramientas que soportan documentación de requisitos en lenguaje que es simultáneamente legible para humanos y ejecutable para pruebas automatizadas (BDD - Behavior-Driven Development).

Una tendencia creciente es la integración directa entre herramientas de gestión de requisitos y herramientas de pruebas. Cuando un requisito está documentado, los casos de prueba pueden vincularse de forma inmediata, permitiendo generar reportes

en tiempo real sobre qué requisitos han sido verificados y cuáles aún presentan brechas de cobertura.

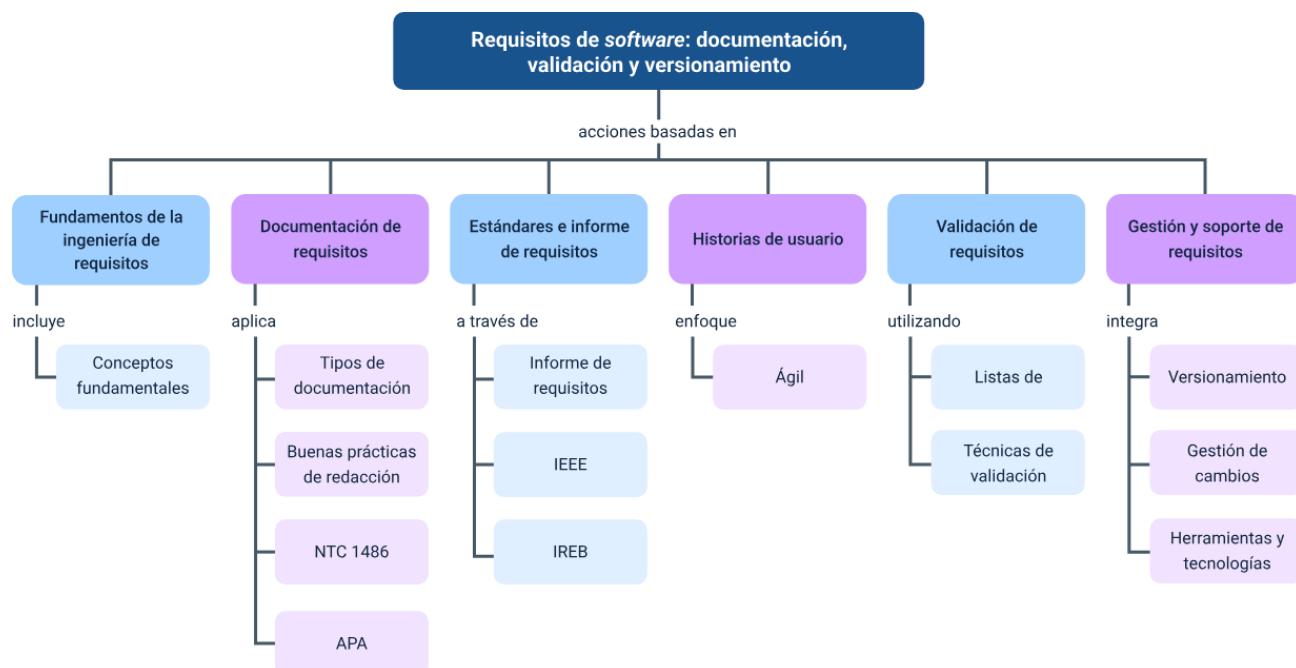
Otra tendencia relevante es el enfoque conocido como requirements as code. En lugar de mantener los requisitos en documentos separados o en sistemas especializados, estos se documentan directamente en el repositorio de código, frecuentemente mediante archivos en formatos como Gherkin. Esto acerca los requisitos a su implementación, facilita su actualización y reduce el riesgo de desalineación entre lo que se define y lo que se desarrolla.

Independientemente de la herramienta utilizada, los principios fundamentales permanecen constantes: centralización de la información, control de versiones, trazabilidad y gestión formal de cambios. Las herramientas no reemplazan estos principios, sino que los automatizan, reduciendo errores, mejorando la visibilidad del proyecto y aumentando la eficiencia del proceso de ingeniería de requisitos.

Síntesis

A lo largo de este componente formativo se analizó la ingeniería de requisitos como una disciplina esencial para el éxito de los proyectos de software. Se comprendió que los requisitos actúan como el puente entre la visión del negocio y la solución técnica y que su correcta definición permite transformar necesidades poco claras en especificaciones implementables y verificables. Se estudiaron distintas formas de documentarlos, desde lenguaje natural hasta modelos y enfoques híbridos, así como los principios de calidad, estándares internacionales y el uso de historias de usuario. Todo ello orientado a reducir errores, facilitar la validación y asegurar que el producto final responda realmente a las necesidades del cliente.

Asimismo, se abordaron prácticas clave como la validación de requisitos, la gestión de cambios y el apoyo de herramientas especializadas que facilitan la trazabilidad y el control del proyecto. Más allá de técnicas o tecnologías, se destacó la importancia de desarrollar una mentalidad crítica y comunicativa, capaz de identificar ambigüedades y alinear a los distintos actores involucrados. Aunque la ingeniería de requisitos continúa evolucionando con nuevas metodologías y tecnologías, sus principios fundamentales permanecen vigentes y constituyen una base indispensable para la construcción de software de calidad y el desarrollo profesional a largo plazo.



Glosario

Análisis de impacto: evaluación sistemática de cómo un cambio propuesto en requisitos afecta otros requisitos, diseño, implementación y pruebas.

Behavior-Driven Development (BDD): enfoque que documenta requisitos en términos de comportamientos observables que pueden ser automáticamente probados.

Compleitud: característica de un requisito que asegura que contiene toda la información necesaria para ser implementado sin información adicional de otras fuentes.

Computer-Aided software Engineering (CASE): herramientas que automatizan aspectos del desarrollo de software incluyendo requisitos, diseño, y pruebas.

Constraint: restricción o limitación que restringe cómo un requisito puede ser implementado, ya sea técnica, organizacional o de negocio.

Design thinking: metodología centrada en el usuario para innovación que enfatiza empatía, experimentación e iteración—valores importantes en elicitación de requisitos.

Documento de visión: documento de alto nivel que describe la visión, scope y beneficios esperados de un proyecto propuesto.

Elicitación: proceso de descubrimiento, extracción y comunicación de requisitos de los stakeholders mediante entrevistas, workshops y análisis.

Encryptación: proceso de transformar información legible en código ilegible sin una clave, requisito crítico para seguridad de datos sensibles.

Entidad: objeto de negocio o concepto importante que el sistema necesita rastrear (cliente, orden, producto en un sistema de ventas).

Granularidad: nivel de detalle de requisitos muy generales (baja granularidad), versus requisitos muy específicos (alta granularidad).

Scrum: Framework ágil para gestión de proyectos que usa historias de usuario, sprints iterativos y reuniones de retroalimentación regular.

Stakeholder: cualquier persona o grupo con interés en el proyecto, incluyendo cliente, usuarios finales, desarrolladores, managers, reguladores.

Waterfall: modelo tradicional de desarrollo de software donde cada fase (requisitos, diseño, implementación, pruebas) se completa antes de la siguiente.

Referencias bibliográficas

Cohn, M. (2004). Historias de usuario aplicadas: Para el desarrollo de software ágil. Wesley Professional.

Davis, A. M. (2003). Gestión de requisitos: lo justo y necesario donde el desarrollo de software se encuentra con el negocio. Dorset House Publishing

IEEE. (2011). IEEE Standard 829-2008: IEEE Standard for Software and System Test Documentation. IEEE Standards Association.

IEEE. (2017). IEEE 830-1998 IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers.

IEEE. (2018). IEEE/ISO/IEC 29148-2018: Systems and software engineering—Life cycle processes—Requirements Engineering. IEEE Standards Association.

Pohl, K. (2010). Ingeniería de requisitos: Fundamentos, principios y técnicas. Springer Publishing Company. (Edición en español).

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Responsable Ecosistema de Recursos Educativos Digitales (RED)	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Solanlly Sánchez Melo	Experta temática	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Juan Daniel Polanco Muñoz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Veimar Celis Meléndez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima

Nombre	Cargo	Centro de Formación y Regional
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima